Worked for 22s ›

Yes — here are the exact Windows steps to generate a fresh EchoVein `.exe`.

Your project already targets both browser and Windows desktop through Electron, and earlier notes say the Electron build should still work with `npm run dist:win`, while also preserving browser launch from `index.html`. Pasted markdown

1. Prepare the project folder

On your PC, make sure your **latest full EchoVein folder** contains at least:

```
index.html
js/
css/
assets/          if present
electron/
  main.cjs
  preload.cjs
package.json
README_WINDOWS_ELECTRON.md
```

Do **not** build from only the patch ZIPs. Apply all patch ZIP contents into your main EchoVein project first.

2. Open PowerShell in the project folder

Example:

PowerShell

```
cd "C:\Users\aless\Desktop\EchoVein"
```

Adjust the path to wherever your project is.

3. Check Node.js is installed

Run:

PowerShell

```
node -v
npm -v
```

You should see versions printed, for example:

```
v22.x.x
```

If PowerShell says `node` or `npm` is not recognised, install Node.js LTS from the official Node.js website, then reopen PowerShell.

4. Do a quick JavaScript syntax check

From the project folder, run:

PowerShell

```
Get-ChildItem .\js\*.js | ForEach-Object { node --check $_.FullName }
```

If you also have Electron files:

PowerShell

```
node --check .\electron\main.cjs
node --check .\electron\preload.cjs
```

No output normally means success. If there is a syntax error, PowerShell will show the file and line number.

5. Install dependencies

Run:

PowerShell

```
npm install
```

This creates or updates:

```
node_modules/
package-lock.json
```

This step may take a few minutes the first time.

6. Test the desktop version before building

Run:

PowerShell

```
npm start
```

Expected result:

```
EchoVein opens in an Electron desktop window.
```

Test quickly:

```
Main menu opens
Start run works
Mission status panel appears
Pressure popup Accept / Ignore works
How to Play opens
No black screen
No console crash
```

Close the game window when done.

7. Build the Windows executable

For a distributable Windows installer:

PowerShell

```
npm run dist:win
```

For a quicker unpacked test build, if your `package.json` supports it:

PowerShell

```
npm run pack:win
```

8. Find the generated EXE

After build, check:

```
dist/
```

You will usually see one or both of these:

```
dist/EchoVein Setup x.y.z.exe
dist/win-unpacked/EchoVein.exe
```

Use them like this:

```
Installer EXE:
Can usually be shared by itself.

win-unpacked:
```

9. Recommended final test

Test both:

```
dist/win-unpacked/EchoVein.exe
```

and, if generated:

```
dist/EchoVein Setup x.y.z.exe
```

Check especially:

```
Start run
Mission Status panel
Hollow Pressure meter
Local Resonance labels
Pressure Objective popup
Ignore Signal button
Accept Risk button
How to Play menu
Controller/menu navigation if available
```

If `npm run dist:win` fails

Run this and send me the output:

PowerShell

```
npm run dist:win
```

Common fixes:

PowerShell

```
npm install
npm install electron electron-builder --save-dev
```

Then retry:

PowerShell

```
npm run dist:win
```

Do not delete `dist/` until you have copied the EXE you want to keep.